DO NOW — RECALL

Floating-point addition

Name: _____ Class: _____ Date: _____

1. Convert 01011000 0011 to denary. Use an eight-bit two’s-complement mantissa and a four-bit two’s-complement exponent.

2. Represent 0.375 in normalised floating-point form using an eight-bit two’s-complement mantissa and a four-bit two’s-complement exponent.

3. Convert 11110110 from 8-bit two’s-complement binary to denary. Show signed place values.

4. Convert AD_{16} to binary, then logically shift the 8-bit value right once. Give the final denary value.

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

5.5 Mantissa 01011000 = 0.6875 ; exponent 0011 = 3 ; $0.6875 \times 2^3 = 5.5$.

2. Model answer

01100000 1111 0.375 = $0.011_2 = 0.11_2 \times 2^{-1}$. Mantissa 01100000 ; exponent 1111 = −1 .

3. Model answer

−10 −128 + 64 + 32 + 16 + 4 + 2 = −10 .

4. Model answer

$AD_{16} = 10101101_2 \rightarrow 01010110_2 = 86$ A logical right shift inserts 0 on the left and discards the rightmost bit.

By the end, you can…

• align exponents while preserving operand values;

• add signed mantissas using sufficient working width;

• normalise a sum and explain possible precision loss.

Vocabulary — know these terms

Recall the term — Exponent alignment

Recall the term — Arithmetic right shift

Recall the term — Sign extension

Recall the term — Guard bits

Recall the term — Renormalisation

Recall the term — Rounding error

NEW LEARNING

Read, annotate and explain

Align → add → normalise

6-bit mantissa:
−1 | ½ ¼ ⅛ 1/16 1/32

4-bit exponent:
−8 4 2 1

Use a temporary extra sign bit for the working sum.

When aligning a negative mantissa, use an arithmetic right shift : repeat the sign bit on the left.

Align the exponents

Choose the larger exponent. Increase the smaller exponent one step at a time and right-shift that mantissa once for each step.

Preserve sign and working precision

An arithmetic right shift repeats 0 for a positive mantissa and 1 for a negative mantissa.

Keep low-order guard bits when a shift would otherwise discard a non-zero bit.

Add using a wider working sum

Sign-extend both aligned mantissas before adding so the calculation cannot wrap into the wrong sign.

Normalise and check

If the sum is too large for the mantissa, shift right and increase the exponent. If redundant sign bits remain, shift left and decrease the exponent.

Explain one step in your own words, using the correct technical terms.

I DO — WORKED EXAMPLE

Follow the worked method

Add 5 + 1.5

Use the common exponent 3

Raise the second exponent from 1 to 3, so shift its mantissa right twice using sign extension.

```
010100 0011
011000 0001 → 000110 0011
```

Use a temporary extra sign bit

```
0010100
 + 0000110
 = 0011010
```

The working sum is $26/32 \times 2^3$.

Return to the requested width

```
0011010
→ 011010 0011
```

The mantissa already begins 01 , so the exponent stays at 3.

Verify in denary

```
5 + 1.5 = 6.5
```

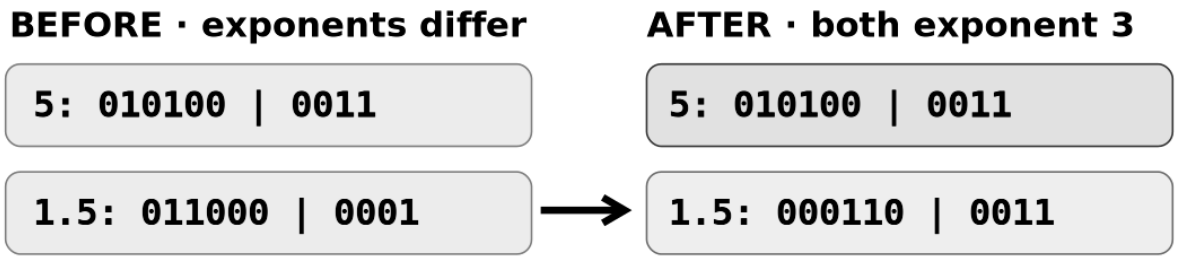
Repeat the method with different inputs in your notebook. Check every intermediate value.

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Add 5 + 1.5

Use a six-bit two’s-complement mantissa and a four-bit two’s-complement exponent. Give the final answer in normalised floating-point form.



Arithmetic right shift ×2 · raise exponent 1 → 3

Mantissa | exponent · each number keeps its value

Align floating-point exponents before adding five and one point five: the second mantissa shifts right twice from 011000 to 000110 while its exponent changes from 1 to 3

WE DO — GUIDED PRACTICE

Try → Discuss → Improve

1. 2. Rescale the sum: 6 + 5

Use a six-bit two’s-complement mantissa and a four-bit two’s-complement exponent. Give the final answer in normalised floating-point form.

Method / working:

2. 3. A negative operand: −5 + 3

Use a six-bit two’s-complement mantissa and a four-bit two’s-complement exponent. Give the final answer in normalised floating-point form.

Method / working:

3. 4. Align a negative mantissa: −3 + (−2)

Use a six-bit two’s-complement mantissa and a four-bit two’s-complement exponent. Give the final answer in normalised floating-point form.

Method / working:

4. 5. Precision loss

Use a six-bit two’s-complement mantissa and a four-bit two’s-complement exponent. State whether an exact normalised floating-point result can be stored.

Method / working:

5. OCR SAMPLE ASSESSMENT · Q7(e) · [3]

Use a four-bit two’s-complement mantissa and a four-bit two’s-complement exponent. Give the final answer in normalised floating-point form.

Method / working:

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: 2. Rescale the sum: 6 + 5 Use a six-bit two’s-complement mantissa and a four-bit two’s-complement exponent. Give the final answer in normalised floating-point form. Build the answer

DISCUSS — Compare intermediate values and your reasoning. Justify one step using the relevant rule, representation or truth table.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

Guided 2: 3. A negative operand: −5 + 3 Use a six-bit two’s-complement mantissa and a four-bit two’s-complement exponent. Give the final answer in normalised floating-point form.

DISCUSS — Compare intermediate values and your reasoning. Justify one step using the relevant rule, representation or truth table.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

Guided 3: 4. Align a negative mantissa: −3 + (−2) Use a six-bit two’s-complement mantissa and a four-bit two’s-complement exponent. Give the final answer in normalised floating-point form.

DISCUSS — Compare intermediate values and your reasoning. Justify one step using the relevant rule, representation or truth table.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

Guided 4: 5. Precision loss Use a six-bit two’s-complement mantissa and a four-bit two’s-complement exponent. State whether an exact normalised floating-point result can be stored.

DISCUSS — Compare intermediate values and your reasoning. Justify one step using the relevant rule, representation or truth table.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

Guided 5: OCR SAMPLE ASSESSMENT · Q7(e) · [3] Use a four-bit two’s-complement mantissa and a four-bit two’s-complement exponent. Give

the final answer in normalised floating-point form.

DISCUSS — Compare intermediate values and your reasoning. Justify one step using the relevant rule, representation or truth table.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Question 1 · OCR OCTOBER 2021 Q2(c) · [5]
Add 011000 0110 and 010100 0100 . Show your working and give your answer in normalised floating-point form using a six-bit two’s-complement mantissa and a four-bit two’s-complement exponent.
Paraphrased exam question; original operands and marks retained. OCR October 2021, Q2(c) .

Question 2

Question 2 · ADAPTED EXAM PRACTICE · [5]
Add 011000 0011 and 010000 0010 using a six-bit two’s-complement mantissa and a four-bit two’s-complement exponent. Show alignment and the widened sum, then give the answer in normalised floating-point form.
Adapted from the OCR addition task: changed values; classroom marks.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

Question 3 · ORIGINAL EXAM-STYLE EXTENSION · [5]
Add 101100 0011 and 011000 0001 using a six-bit two’s-complement mantissa and a four-bit two’s-complement exponent. Show alignment and signed addition, then give the answer in normalised floating-point form.

Question 4

Question 4 · ORIGINAL EXAM-STYLE EXPLANATIONS · [5]
Why must exponents match before mantissas are added directly? [2]
What enters from the left when a negative mantissa is shifted right for alignment, and why? [2]
Why can addition lose precision even if the result is within exponent range? [1]

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

Floating-point mantissas must be aligned to the same exponent before they are added directly.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

Build the answer

IMPROVE: Explain one correction to your own reasoning.

Guided 2 — original CHECK

Build the answer

IMPROVE: Explain one correction to your own reasoning.

Guided 3 — original CHECK

Build the answer

IMPROVE: Explain one correction to your own reasoning.

Guided 4 — original CHECK

Build the answer

Use the rounding or truncation rule specified by the question.

IMPROVE: Explain one correction to your own reasoning.

Guided 5 — original CHECK

Build the answer

IMPROVE: Explain one correction to your own reasoning.